

Estimating Agrarian Land Efficiency

1 Introduction

Problem: To examine the efficiency of agricultural fields. These classifications determine how much a field is fulfilling its potential-to-date, based on resources, dimensions, and other characteristics. A field classified as “low performing” may still produce a decent yield, but due to inefficiencies such as distance from the market it is not maximizing its potential.

Goal: The objective is to create a classifier which can predict whether an agricultural field is “high performing,” “moderately performing,” or “low performing,” depending on the test set. The classifier is evaluated on the **macro F1-metric**, which weights each of the three classes equally and therefore rewards balanced performance across all of them rather than accuracy on the majority class alone.

Dataset Overview: The train set consists of 112,569 samples, each uniquely identified by a UID. The target label **Target** indicates the performance level and can take one of three values: “low,” “medium,” or “high.” The test set comprises 15,921 samples with unique UIDs disjoint from the training set. This dataset assesses the efficiency of agricultural field usage, classifying each field.

2 Data Preprocessing

2.1 Data Type and Dataset Dimensions Analysis

To understand which data types are associated with each column in the training dataset and ensure proper handling during preprocessing and modeling stages. We also measured the dimensions of the dataset to get an understanding of the data’s scale and the number of features that need examination.

2.2 Statistical Information

Calculated summary statistics on the numerical columns to identify columns with low variance, which displays key statistics like mean, standard deviation, minimum, maximum, etc.

Observations: Some columns have a standard deviation close to zero, indicating near-zero variance and quasi-constant features. Features with near-zero variance are unlikely to contribute meaningfully to the model’s predictive power and were flagged for potential exclusion.

2.3 Determination of Quasi-Constant Features

Identified quasi-constant columns, where values appear to be equal in more than 99% of rows in the dataset.

Observation: No quasi-constant columns were found.

2.4 Redundant Rows

Identified duplicated rows in the dataset to eliminate redundant information that could skew model training.

Observation: No duplicated rows were present in the dataset. Therefore, no methods for removing duplicates were applied.

3 Test–Train Splitting

3.1 Feature versus Target Separation

Extracted the target variable from the feature variables for model training. This separation ensures that only the feature data is used to train the model, while the target data provides supervision for learning.

3.2 Target Encoding

Converted the target variable to a numeric scale, so it is compatible with model algorithms.

3.3 Observations

The target variable was encoded with integer values assigned to each class:

- Low $\rightarrow$ 0
- Medium $\rightarrow$ 1
- High $\rightarrow$ 2

This encoding enables direct application of classification algorithms that require numeric target labels.

4 Exploratory Data Analysis

4.1 Missing Values Analysis

Measured the percentage of missing values across attributes to guide strategies on imputation or exclusion.

Observation: The calculated missing percentages showed data loss per feature, forming a basis for deciding whether to impute or exclude each column.

4.2 Removing Columns with Too Many Missing Values

Removed features with more than 90% missing values, as they contribute limited useful information.

Observation: Dropped columns with extensive missing values, reducing feature dimensionality and improving dataset quality. A list of these columns and their count was printed for verification.

4.3 Eliminating Rows with Insignificant Counts of Missing Values

Cleaned the dataset by removing rows belonging to columns with fewer than 2% missing values.

Observation: Retained rows with minimal missing values, thereby maintaining data integrity and enhancing quality while minimizing data loss.

4.4 Correlation Analysis

Analyzed feature inter-relationships to understand dependencies and identify highly correlated features.

Observation: The heatmap depicted the intensity and direction of correlations between feature pairs, highlighting dependencies that may require further investigation or feature engineering.

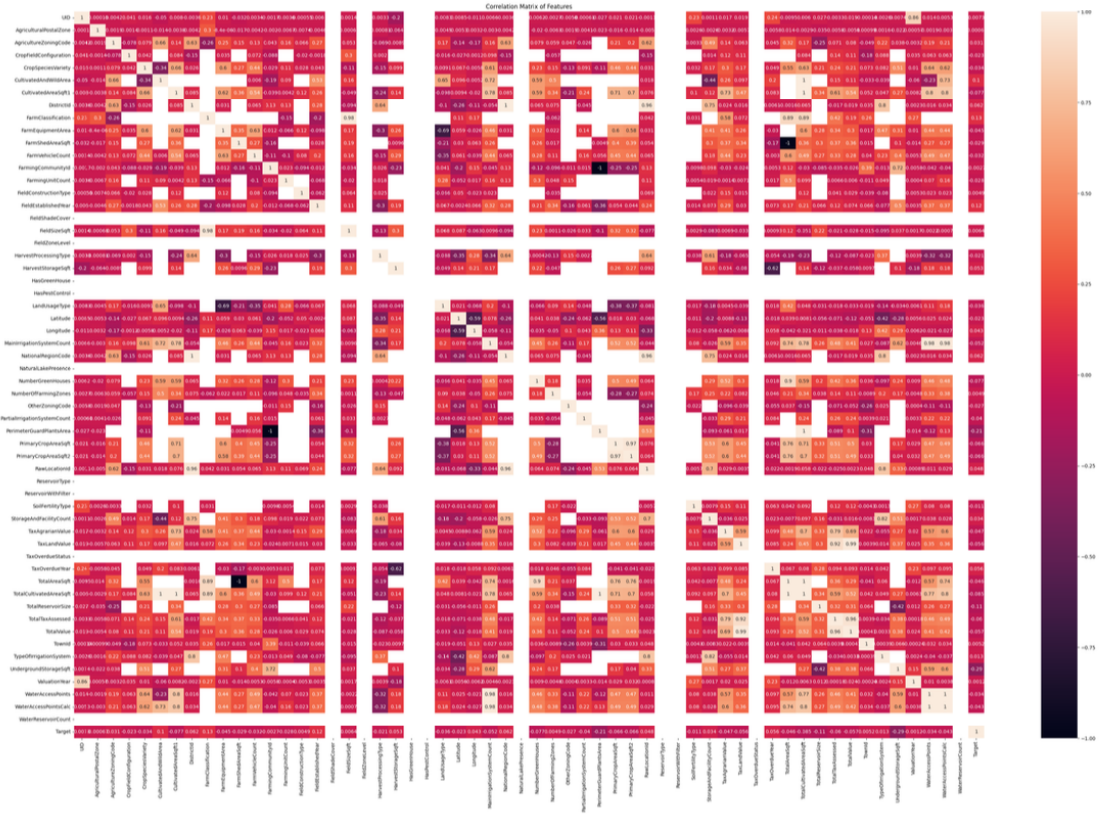


Figure 1: Correlation among features

4.5 Handling Missing Values by Imputation

Group-wise imputation was applied to both numerical and categorical features. In every case the statistics were computed on `X_train` only and then reused on `X_test`, so that the two sets remain consistent and no information leaks from the test data.

Numerical features

- **CultivatedAreaSqft1:**
 - The missing values were iteratively filled based on the group-wise mean within `TotalCultivatedAreaSqft`, `MainIrrigationSystemCount`, and `TaxAgrarianValue`.
 - Remaining missing values were filled with the mean of `CultivatedAreaSqft1`.
 - The same group-based and mean strategy was applied to `X_test`, using values derived from `X_train`.
- **FieldSizeSqft:**
 - Missing values were imputed using the mean for each `TownId` within the `X_train` dataset.
 - Remaining missing values were imputed with the overall mean of `FieldSizeSqft` from `X_train`.
 - The same strategy was applied to `X_test`.

Categorical features

- **SoilFertilityType:**

- Imputed using the mode of `SoilFertilityType` from `X_train`.
- The same mode applied to `X_test`.
- **HarvestProcessingType:**
 - Missing values were filled iteratively based on the mode value (the most common) within `RawLocationId`, `NationalRegionCode`, and `DistrictId`.
 - Remaining missing values were filled using the overall mode of `HarvestProcessingType`, determined from `X_train`.
 - This approach was applied to `X_test` by using mode values derived from `X_train`.

Observation: This group-wise imputation approach exploits similarities among geographic and administrative groups to make more accurate imputations than a single global statistic would allow, while keeping `X_train` and `X_test` consistent.

4.6 Histogram Analysis

We analyzed the distribution of various numerical features in the dataset through histograms. The features selected for analysis are based on their numerical nature and are vital for understanding the dataset’s underlying trends and structure. This helped in subsequent preprocessing, feature selection, and modeling stages.

Observation: By analyzing the distribution of numerical features, we got a clearer understanding of how the data is structured. Features with skewed distributions may require transformation or normalization, while others may be resolved from outlier handling.

4.7 Spatial Distribution of Missing Values

Plotting longitude and latitude against `FieldEstablishedYear` shows where the missing records sit geographically, before and after imputation. The red points in Figure 2 mark records with a missing establishment year; they are spread across the whole region rather than concentrated in one district, which supports the use of group-wise imputation. Figure 3 shows the same plot once the values have been filled in.

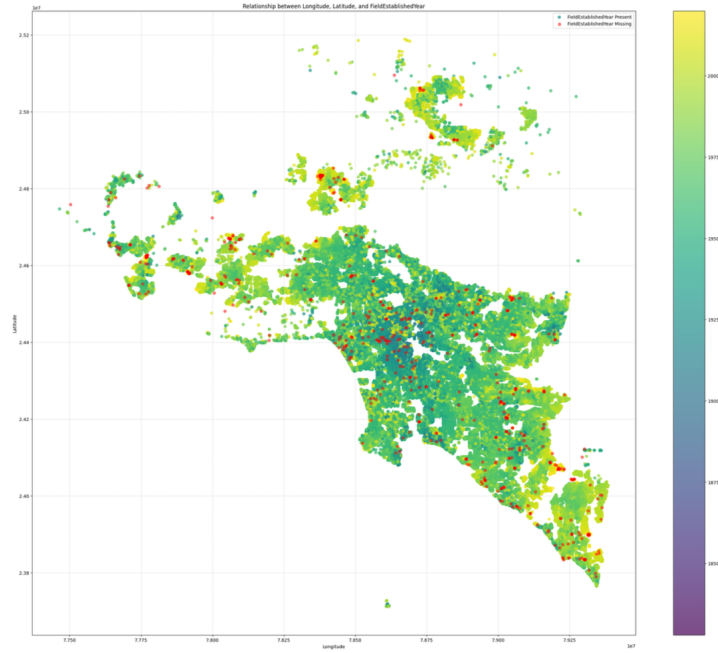


Figure 2: Graph with null values

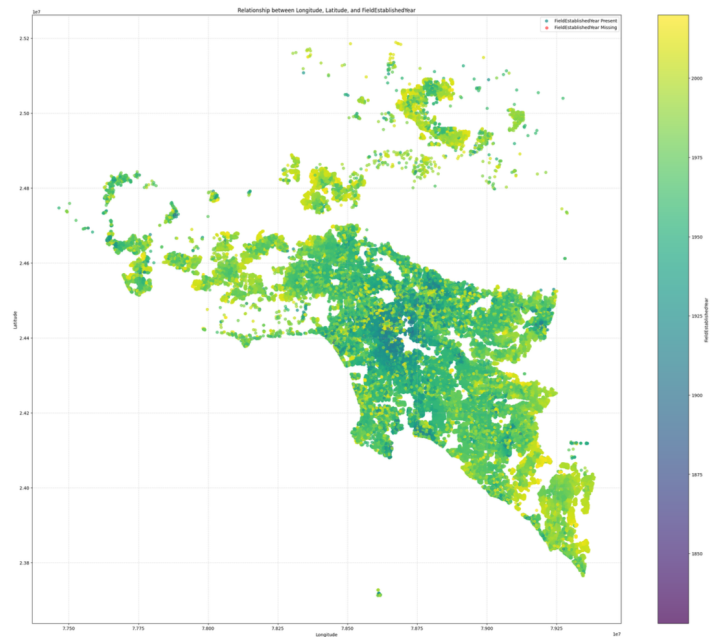


Figure 3: Graph after imputing null values

5 Feature Engineering

5.1 Feature Generation

(a) Farm Age

We calculated the farm's age based on the difference between the valuation year

(`ValuationYear`) and the field establishment year (`FieldEstablishedYear`):

$$\text{FarmAgeYears} = \text{ValuationYear} - \text{FieldEstablishedYear}$$

Observation: This feature provides an indication of how long the farm has been established, which could be relevant for assessing farm development and maturity.

(b) Tax Burden and Agrarian Value Ratios

We introduced two new features to capture the tax burden and agrarian value proportion:

$$\text{TaxBurden} = \frac{\text{TotalTaxAssessed}}{\text{TotalValue} + 1}$$

$$\text{AgrarianValueProportion} = \frac{\text{TaxAgrarianValue}}{\text{TotalValue} + 1}$$

Observation: These features provide insights into the relationship between tax assessments and the total value of the farm, which may help in predicting the financial health of the farm.

(c) Total Irrigation Systems and Density

We created a new feature `TotalIrrigationSystems` by considering both the main and partial irrigation systems:

$$\text{TotalIrrigationSystems} = \text{MainIrrigationSystemCount} + 0.5 \times \text{PartialIrrigationSystemCount}$$

Additionally, we introduced the `IrrigationAreaDensity` feature to assess the irrigation density relative to the cultivated area:

$$\text{IrrigationAreaDensity} = \frac{\text{TotalIrrigationSystems}}{\text{TotalCultivatedAreaSqft}}$$

Observation: These features provide valuable insights into the water management infrastructure of the farm.

(d) Water Resources and Density

We combined multiple variables to create a comprehensive measure of water resources available on the farm:

$$\begin{aligned} \text{WaterResources} = & \text{WaterAccessPoints} + \text{WaterReservoirCount} \\ & + 0.5 \times (\text{NaturalLakePresence}) \times (\text{WaterAccessPoints} + \text{WaterReservoirCount}) \end{aligned}$$

Next, we calculated the `WaterAreaDensity` to understand how well the water resources are distributed relative to the cultivated area:

$$\text{WaterAreaDensity} = \frac{\text{WaterResources}}{\text{TotalCultivatedAreaSqft}}$$

Observation: These features give us insights into water availability and its relationship with farm area.

(e) Field Utilization

We created a new feature `FieldUtilization` to assess the efficiency of the farm’s resources. It combines the derived quantities above with soil fertility, vehicles and storage, all normalized by field size:

$$\begin{aligned} \text{FieldUtilization} = & \frac{\text{SoilFertilityType} + \text{WaterResources} + \text{FarmVehicleCount}}{\text{FieldSizeSqft}} \\ & + \frac{\text{TotalIrrigationSystems} + \text{StorageAndFacilityCount}}{\text{FieldSizeSqft}} \end{aligned}$$

Observation: This feature provides an overall measure of how effectively the farm utilizes its resources for crop production, and it maps directly onto the notion of “potential achieved” that the target label encodes.

5.2 Dealing with Highly Correlated Features and Imputing Remaining Null Values

Calculated the correlation matrix, and the absolute values of correlations were utilized to determine relationships between columns of the original dataframe. For each column of `X_train` that has missing values, those missing values are filled with the mean calculated from the corresponding column in `X_train`. To ensure consistency between the training and test datasets, the same mean value calculated from `X_train` was used to replace missing values for the corresponding columns within `X_test`.

Observation: This eliminated some redundancy and prevents overfitting by using a technique that identifies columns with high correlation.

5.3 One-Hot Encoding of Categorical Data

Categorical variables such as `LandUsageType`, `HarvestProcessingType`, `TypeOfIrrigationSystem`, and `SoilFertilityType` were one-hot encoded to convert them into numerical representations suitable for the model, in both the train and test set.

5.4 Feature Selection

(a) Random Forest Feature Importance

Fit the model to `X_train` and take out the feature importance scores. The score was higher for the features that showed higher importance.

(b) Mutual Information

Mutual information was calculated to quantify the degree of dependence of individual features with respect to the target. Features that showed high mutual information values were assigned higher importance scores.

(c) Lasso Feature Selection

Lasso regression was used to determine the significant features by penalizing the coefficients of the less crucial variables. The coefficients were standardized to create scores; high values indicated important features.

Observation: The top n features were selected using combined scores obtained from the three different methods. This way, the feature selection algorithm ensures that the most relevant features essential for prediction of the target variable are retained.

5.5 PCA

Although PCA was applied for dimensionality reduction to simplify the dataset, the model's accuracy decreased after the reduction in dimensionality. As a result, PCA was removed from the process to retain the full feature set for better performance.

6 Observations

- **High Missing Values in Many Features:**

The dataset consists of a high degree of missing values across various features, necessitating preprocessing to maintain data quality and improve model accuracy. Columns with 90% or more null values were dropped, as they contribute minimal information and may not be reliable for imputation.

- **Low Missing Values in Certain Columns:**

Columns with less than 2% missing values had their rows with null values dropped. This

approach minimizes data loss while retaining valuable information across most of the dataset.

- **Key Columns with Few Missing Values:**

Features such as `RawLocationId`, `NationalRegionCode`, and `DistrictId` have a low degree of missing values. Given their relatively complete data, they are suitable candidates for imputing missing values in other features, potentially assisting in filling gaps based on geographic or administrative associations.

- **Useful Attributes for Imputation:**

Attributes like `MainIrrigationSystemCount`, `TotalCultivatedAreaSqft`, and `TaxAgrarianValue` also have minimal missing values. These features could provide reliable information for imputing other missing data points, especially in attributes related to land use, tax values, or irrigation systems.

- **Multi-class target:**

The target variable, `Target`, has 3 unique values, making it suitable for a classification task. This allows us to approach the problem as a multi-class classification, where we predict the category or class for each record based on the provided features.

7 Model Training

A **Balanced Random Forest Classifier** is implemented to handle the class imbalance in the dataset. Because the submission is scored on the macro F1-metric, which treats all three classes equally, balancing the class weights during training matters as much as raw accuracy.

The model is trained using the selected features from the training dataset (`X_train_selected`) and the target variable (`y_train`).

Once trained, the model was used to make predictions on the test dataset (`X_test_selected`). The resulting predictions (`y_pred`) provide the performance classification of each field as “high,” “medium,” or “low” performing.

A new DataFrame (`result_df`) is created to store the UID (unique identifier) from the test set along with the corresponding predicted target labels. This DataFrame is constructed using the UID column from `X_test` and the `y_pred` mapped values. Finally, the DataFrame is saved to a CSV file, without including the index.

8 Outcome

The final result is a CSV file containing the predictions for each test sample, with the target labels mapped to “low”, “medium”, or “high”. This file can be used for further analysis or evaluation of the model’s performance on the macro F1-metric.